JOHANNES KEPLER
UNIVERSITY LINZ

Author
Jakob Waldhäusl

Submission
Institute for System Software (SSW)

Thesis Supervisor
Dipl.-Ing. **Thomas Schrott**, BSc

August 2026

Performance Analysis of Data-oriented Programming in Rust



Bachelor's Thesis

to confer the academic degree of

Bachelor of Science

in the Bachelor's Program

Computer Science

JOHANNES KEPLER UNIVERSITY LINZ
Altenberger Straße 69
4040 Linz, Austria
jku.at

Abstract

< Abstract in english >

Kurzfassung

< Abstract auf Deutsch >

Contents

Abstract ii

Kurzfassung iii

1 Introduction 1
- 1.1 Motivation . . . 1
- 1.2 The Shift to DOP . . . 1
- 1.3 Objectives and Methodology . . . 2

2 Background 3
- 2.1 Data-oriented Programming . . . 3
- 2.2 Caching and CPU-Architecture . . . 5
- 2.3 Data-Level Parallelism via SIMD . . . 5
- 2.4 Entity Component Systems . . . 5
- 2.5 Rust and Bevy engine . . . 7

3 Benchmark Design and Implementation 8
- 3.1 Simulation . . . 8
- 3.2 Evaluated Architectures . . . 9
- 3.3 Scenario 1: Compute-Bound . . . 12
- 3.4 Scenario 2: Memory-Bound . . . 15
- 3.5 Scenario 3: Allocation and Deallocation . . . 17
- 3.6 Execution and Configuration . . . 19

4 Evaluation 21
- 4.1 Methodology . . . 21
- 4.2 Scenario 1: Compute-Bound . . . 23
- 4.3 Scenario 2: Memory-Bound . . . 26
- 4.4 Scenario 3: Allocation and Deallocation . . . 28
- 4.5 Discussion . . . 28
- 4.6 Limitations . . . 29

5 Related Work 30

6 Future Work 31

7 Conclusion 32

Bibliography **33**

List of Tables **36**

List of Figures **37**

List of Acronyms **38**

Chapter 1

Introduction

1.1 Motivation

In the past decades, processor speeds have vastly increased due to increased clock frequency and higher core counts, while DRAM has not been able to keep up. This phenomenon is colloquially known as the ”memory wall”. The resulting problem is that execution speed may no longer be limited by processing power, but by the processor having to wait for data from the DRAM [6]. To address this gap, modern software development has to take into consideration the underlying hardware constraints.

1.2 The Shift to DOP

Object-oriented Programming (OOP) has enjoyed wide adoption and is usually taught long before Data-oriented Programming (DOP) in most curricula. However, OOP poses challenges when it comes to cache-optimization, for example, due to objects bundling data and logic, which leads to inefficient memory layouts and cache pollution [8]. More modern approaches, such as DOP, aim to mitigate challenges such as these by decoupling logic and data. However, there are more mechanisms that DOP employs to improve cache efficiency and execution time, some of the more significant ones being Single Instruction, Multiple Data (SIMD) vectorization (Section 2.3) and use of the Entity Component System (ECS) software architecture design pattern (Section 2.4).

1.3 Objectives and Methodology

The aim of this thesis is to measure the difference in execution time between OOP and DOP approaches, as the exact difference is hardly ever quantified. The created benchmarks aim to reflect realistic use cases and lean towards the gaming industry, as there has been a significant push in recent years to adopt the ECS pattern. Rust will act as the programming language with the Bevy engine providing the framework. Rust provides memory safety without requiring a garbage collector [13]. The absence of garbage collection is especially desirable for benchmarking, as it eliminates nondeterministic collection pauses that would otherwise distort time measurements. Bevy builds directly on these properties. According to its documentation, Bevy ECS represents components as plain Rust structs and systems as ordinary functions, which the project contrasts with other Rust ECS implementations that often rely on more complex constructs such as lifetimes or builder patterns [3]. This keeps the framework-based implementation close in form to the hand-written architectures it is compared against, which is desirable when the comparison is meant to isolate memory layout rather than API design.

Chapter 2

Background

This section describes the relevant parts of DOP, which also include SIMD vectorization and the ECS design pattern, as they are integral to the nature of the systems under inspection. Furthermore, a brief overview of Rust and the Bevy engine is presented.

2.1 Data-oriented Programming

The OOP paradigm undoubtedly has many useful features such as encapsulation, polymorphism and inheritance. These features enable software developers to think of problems in terms of objects and their interactions, with objects ideally being self-contained. Due to this, programmers are able to easily reuse and maintain them without having to fear side effects [2]. Nevertheless, objects encapsulating the entirety of their data and logic is a double-edged sword.

For example, a program that solely updates the position of some objects, where each object also holds further attributes, e.g. color or weight. Due to encapsulation, the CPU has to load all the data of these objects into its

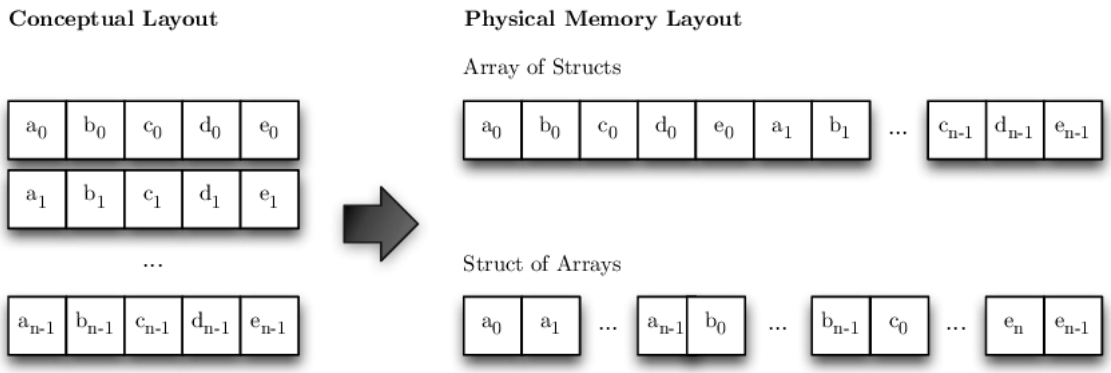


Figure 2.1: Memory Layout Visualization: AoS vs. SoA (taken from [17])

cache, even though it should only have to load the location for its computation. This phenomenon is often called ”cache pollution” [6] which poses a problem due to another phenomenon called the ”memory wall”, which describes the growing gap between processor and memory speeds: as processor speed increases faster than memory speed, an increasing number of programs becomes limited not by clock frequency but by the time spent waiting for data from memory. Over the past decades, processor performance has improved far more quickly than memory speed. Wulf and McKee [20] illustrated this disparity in 1995, reporting an annual increase of roughly 80 % in processor speed against only 7 % for memory. While the exact rates have changed since, the underlying trend has persisted, and the resulting gap has become one of the biggest bottlenecks in high-performance computing, as the CPU often stays idle while waiting for data to be fetched from main memory [10].

One of the most promising solutions to these problems is DOP, as it aims to decouple data from logic and thus reduce cache pollution and the CPU idling [7]. DOP improves CPU utilization by fundamentally changing how data is handled by moving away from the classical OOP memory layout called Array of Structures (AoS) and instead using a more optimized Structure of Arrays (SoA) layout [7].

In Figure 2.1, both memory layouts are compared. On the conceptual level, there are simply structs with data, e.g. objects with fields. The more conventional AoS layout stores these structs sequentially within an array, thus the name. In contrast, the SoA layout creates multiple arrays which contain only one type of field. As an example, one could want to manage 1000 `Person` objects which possess the fields `String name`, `float weight` and `Color hairColor`. The AoS layout would result in a `Person` Array with 1000 entries. Contrarily, the SoA layout would result in three arrays, one of type `String` containing the `name` values of all 1000 `Person` instances. Similarly, an array is created for `weight` and `hairColor`.

This is due to CPU caches exploiting a principle called spatial locality. Drepper argues [6] that since neighboring data is likely to be used together, it should be loaded into the cache together. This is what the entire idea of SoA is based upon. If one wants to perform a read or write operation on only the `name` of these 1000 `Person` instances, the CPU can load much more relevant data than with the AoS layout, as the cache is not clogged by the irrelevant `weight` and `hairColor` data. Thus, while the SoA layout might appear confusing to programmers used to the AoS layout, it is necessary in order to properly achieve the performance gains granted by SIMD vectorization.

2.2 Caching and CPU-Architecture

2.3 Data-Level Parallelism via SIMD

Traditionally, to perform operations on data, the CPU has to load these values into a General Purpose Register (GPR). These usually have a fixed size of 32-bit or 64-bit, meaning an instruction can operate on two values of this size at once. For example, assuming a 32-bit environment, a scalar add instruction can add no more than two 32-bit values per cycle.

In order to overcome this limitation, modern CPUs contain specialized SIMD vector units. These units use much wider registers, such as the 512-bit registers found in the Intel MIC architecture, which allow the CPU to treat a single register as a vector of values. By using 512-bit registers, operations can be performed on up to 16 single-precision or 8 double-precision floating-point values in parallel [19]. On consumer grade CPUs, the AVX2 instruction set is widely adopted, using 256-bit registers.

This effectively parallelizes the computation and significantly increases throughput, under the condition that the data is laid out in memory as explained in Section 2.1. When looking at the memory layouts in Figure 2.1 it becomes clear why SIMD vectorization can only bring significant performance gains when the data is stored in a SoA layout. If the data was instead organized as in the AoS layout, SIMD units could not be fed with packed, homogenous data, causing the cache to be filled with irrelevant data that hinders the aspired parallel execution of operations. To achieve the outlined hardware efficiencies without the programmer having to manually manage memory, the ECS design pattern may be employed.

2.4 Entity Component Systems

As the presentation by Bilas [5] at the Game Developers Conference in 2002 shows, there has been a desperate need for the ECS pattern since at least the early 2000s. He explains that with increasing complexity of game systems, the resulting hardcoded class tree must undergo repeated refactoring and will naturally stray further and further from clean OOP principles. He noticed that when re-examining the problem of complex object schemes or structures, that he could also view it as a database and that in the end, only the data matters.

This data-centric perspective is the essence of the ECS pattern. It encourages clean separation of data and logic by employing a 3-level architecture as can be seen in Figure 2.2. (1) Entities can be viewed as an identifier value to which components may be attributed. (2) A Component is a domain-specific datatype and its value is comparable to the field value of an object. One entity may possess multiple components. (3) Lastly, a system is a domain-specific piece of logic or behavior. This system performs computations on entities and their components. Entities and Components represent a game state, with the Systems performing the transitions between states [18].

As an example, we revisit the `Person` objects from Section 2.1. We discussed that the SoA memory layout would greatly improve performance, but we did not explain how this layout would be achieved. Traditionally, one would define a `Person` class with fields and methods and then instantiate objects. To transform this into a SoA memory layout, the ECS pattern is applied as follows:

- Entities are created as unique identifiers.
- Components such as `name`, `weight` and `hairColor` are associated with these entities. In an archetype based ECS like Bevy, this ensures that all `weight` components are stored in a contiguous array, separate from `hairColor` [4].
- Lastly, Systems are defined to perform transformations by querying for entities that possess a certain set of components. For example, updating the `hairColor` component of all `Person` instances that possess the `name` and `hairColor` components. This allows the system to iterate over a tightly packed array of only relevant data, enabling the use of SIMD instructions.

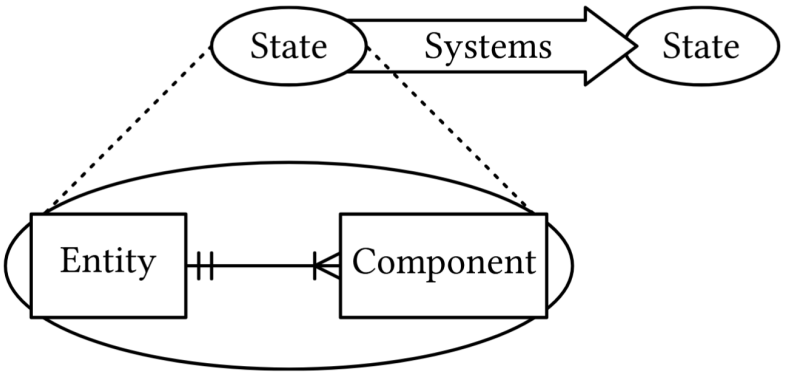


Figure 2.2: Visualization of the ECS pattern depicting (1) Entities, (2) Components and (3) Systems (taken from [18])

2.5 Rust and Bevy engine

Rust is a systems programming language which provides performance comparable to C++ and memory safety through its unique ownership system. The compiler enforces strict rules on how data may be accessed and modified at compile-time. By doing so, Rust eliminates entire classes of bugs such as data races, and without requiring a garbage collector [13].

The Bevy engine expands upon this foundation, using an archetype-based ECS framework that prioritizes performance and modularity. Archetypes group entities with identical combinations of components, which enforces the SoA memory layout [4].

Slices

Chapter 3

Benchmark Design and Implementation

This chapter describes the benchmark program developed as part of this thesis, which is used to measure the performance of two object-oriented and two data-oriented architectures. First, it introduces the simulation on which all performance measurements are based. Then, the four architectures and the three scenarios in which they are measured are defined. The chapter concludes with the configuration and execution of an individual benchmark run.

3.1 Simulation

To properly analyze the performance difference between DOP and OOP, the used simulation must be conceptually and mathematically identical across all evaluated architectures within a given scenario. Otherwise, differences in performance cannot be reliably attributed to the effects of adhering to one paradigm or the other, as they could just as likely occur due to inhomogeneous workloads. Additionally, the simulation must provide a workload that is non-trivial enough to cause the property under inspection to dominate the runtime, while still exercising that property. For example, a complex calculation may dominate the run time in a compute-bound scenario, yet it does not exercise the memory hierarchy sufficiently to expose the differences between memory layouts.

The benchmark simulation performs gravitational calculations on a certain number of planets. These planets always have the same three conceptual attributes, a (1) 3D position in space, a (2) 3D velocity and (3) a payload that acts as a way to artificially inflate the size of the planet objects. The position and velocity of all planets are randomly generated during the setup phase with a seeded random number generator, while

the payload is a fixed size set at compile time. This design enables all three scenarios to operate on the same underlying simulation, with each of them varying a different parameter, e.g., planet count or payload size.

Following the laws of Newtonian gravitation [14, p. 85], the simulation assumes that all planets are accelerated towards a central body of a large fixed mass (e.g., a black hole) at the origin. Due to the planets being accelerated towards this central mass, the planet's own mass cancels out, so the acceleration depends only on its position relative to that mass. The acceleration therefore simplifies to Equation 3.1,

$$\vec{a} = \frac{G}{r^2} \cdot \hat{d} \tag{3.1}$$

where r is the distance between the planet and the center, and $\hat{d}$ is the unit vector pointing from the planet towards the center. Here, G is not the physical gravitational constant but a fixed scaling factor that includes the constant central mass. Its value is derived from the gravitational constant $6.674 \times 10^{-11}\,\mathrm{N\,m^2\,kg^{-2}}$ scaled by a factor of 10^{14}, i.e., $G \approx 6674$, so that the resulting accelerations remain in a numerically well-behaved range for 32-bit floating-point arithmetic. The planet's velocity and position are then integrated over a fixed time step Δt.

3.2 Evaluated Architectures

Chapter 2 contrasted OOP and DOP on the level of paradigms. However, a paradigm itself cannot be benchmarked. Concepts such as encapsulation or a data-oriented mindset leave no direct trace in hardware. What reaches the hardware is the resulting arrangement of data in memory and the pattern in which that data is accessed, which can then be measured. Thus, the comparison is reduced to representative implementations that differ precisely in these two properties. Since the simulated workload itself is identical across all of them (see Section 3.1), any measured difference can be attributed to the memory layout rather than to the workload. In total, four architectures are evaluated, two of which represent OOP and two of which represent DOP design principles. They are arranged such that each of them differs from the preceding one in primarily one property, namely memory layout, grouping of fields and usage of a higher-level ECS. Thus, this arrangement spans a range from maximum indirection to maximum contiguity.

(1) Pointer Chasing (PC) The PC architecture stores planets as `Vec<Box<PlanetAoS>>`, where `PlanetAoS` has three fields: a `Vec3` for the planet's velocity, another `Vec3` for its position, and an array `[u8; PAYLOAD_SIZE]` for the payload. Every planet is allocated individually on the heap, with the vector solely holding the pointers to said planets. In Rust, this indirection is provided by `Box<T>`, where the content of the box lives on the heap and the box itself is a pointer of known size referring to the content [13]. This architecture represents the reference semantics of common object-oriented languages such as Java or C#, where objects are allocated individually and accessed through references.

Since caches exploit spatial locality [6], this layout is the least favorable one under inspection, as traversing the collection means following a pointer per element instead of having the data be laid out in a contiguous manner. This traversal pattern is commonly referred to as pointer chasing with Drepper [6] using it to demonstrate the limits of hardware prefetching. However, when objects are created within a single uninterrupted loop, a general-purpose allocator may place them in near-contiguous memory, in which case the architecture would degenerate into the AoS architecture described below. To prevent this and ensure the layout causes pointer chasing, the vector is shuffled once after setup. This mirrors a long-living application under manual memory management, such as in C++, whose heap becomes fragmented as objects are allocated and freed over time. Managed languages with compacting garbage collectors, such as Java or C#, relocate live objects and thus reduce such fragmentation, so the shuffled layout represents the worst case for those runtimes rather than their typical state.

(2) Array of Structures (AoS) The AoS architecture stores the identical `PlanetAoS` struct in a `Vec<PlanetAoS>`, i.e., without any indirection. The planets are laid out contiguously within the vector, whereas the fields of a single planet remain interleaved, as illustrated in Figure 2.1: position, velocity and payload of one planet are stored adjacently, followed by the fields of the next planet.

Within the object-oriented paradigm, this architecture therefore represents the most contiguous layout achievable. The indirection present in the PC architecture is removed while retaining the conceptual properties of an object, namely that all of its data is kept together. As the AoS and PC architectures share the exact same `PlanetAoS` struct and differ solely in the presence of a `Box<T>`, any measured difference can be directly attributed to the indirection alone.

(3) Structure of Arrays (SoA) The SoA architecture, in contrast to the previous two, adheres to DOP principles. Instead of one vector holding whole planets, one vector is created per individual field. This results in three vectors for the x, y, and z coordinates of a planet's position, three vectors for the x, y, and z values of a planet's velocity and a vector holding the payload. Thus, a planet is no longer a singular entity with inhomogeneous data, but an index shared across seven homogeneous vectors.

This is the layout as described in Section 2.1 and represents the data-oriented ideal. It is implemented manually without any framework, so that it serves as the performance baseline against which any abstraction built on top of a SoA layout is measured. Any performance costs incurred by abstraction built on top of a SoA layout are measured in relation to this baseline. However, manually managing and synchronizing seven parallel vectors is error-prone and cumbersome and does not reflect how data-oriented code is written in practice, where such layouts are typically managed by a framework rather than by hand. This motivates the fourth architecture, which adopts the Bevy ECS framework.

(4) Bevy ECS The Bevy architecture represents the same idea as the SoA architecture, but implemented with an ECS framework. Bevy groups entities, in this case planets, by archetypes, i.e., by their exact set of components [18], and stores the components of an archetype in contiguous, per-component storages [4]. Since all planets in this simulation possess an identical combination of components (`Position`, `Velocity`, `Payload`), they occupy the same archetype and the resulting storage pattern is thus conceptually equivalent to that of the SoA architecture.

Therefore, the difference to the SoA architecture is not how the data is stored, but the abstraction required to keep such a data-oriented layout maintainable. This includes entity identifiers, archetype metadata, and especially the query mechanism through which components are accessed. The idiomatic query iterator yields only one entity's components at a time, which could hinder vectorization. Thus, `contiguous_iter_mut` is used instead of `iter_mut`, as it returns slices of homogeneous component blocks on which the compiler may perform vectorization. A benchmark accompanying the feature confirms this: idiomatic iteration shows no AVX2 speed up, whereas contiguous slices do [12]. As it is not yet part of a released engine version, the function is taken from an in-development build [12].

Therefore, Bevy represents DOP as it is realistically adopted and the comparison with the SoA architecture quantifies how much that convenience costs.

Vector Types and Comparability These four architectures do not only differ in memory layout, but also in their representation of position and velocity. While PC and AoS store them as `Vec3`, i.e., three consecutive `f32` values, the SoA architecture stores the individual coordinates in separate vectors of `f32`. The Bevy architecture uses `Vec3A`, which extends the three `f32` values by four bytes of padding and is aligned to a 16-byte boundary [9]. A single position or velocity value therefore occupies 16 bytes instead of the 12 bytes occupied by the three coordinates in the remaining architectures. This is a deliberate decision, as each architecture uses the representation that is idiomatic for it. The measurements thus reflect the performance of code as it would be written in practice, instead of code that has been homogenized for the sake of uniformity.

Enforcing a single representation would distort the comparison in one direction or the other. Forcing Bevy to use `Vec3` would deny it the aligned accesses that vectorized instructions rely on, since 16-byte memory accesses are expected to be 16-byte aligned and the instruction variants handling unaligned accesses are slower [6]. Conversely, forcing `Vec3A` onto the PC and AoS structs would pad them and therefore inflate the memory footprint the comparison measures. This does not apply to SoA, which stores the coordinates as raw `f32` and thus has no vector type to align in the first place.

The consequence of this decision is that the amount of hot data per planet is not identical across all architectures. Except for Bevy, which carries 32 bytes of position and velocity per planet, all architectures carry 24 bytes. Bevy therefore trades a larger data footprint for the alignment its layout exploits. The payload, in contrast, is identical across all architectures and, when varied, is varied for all four simultaneously. This is relevant as it is one of the parameters varied in Scenarios 2 and 3.

3.3 Scenario 1: Compute-Bound

The first scenario aims to answer the question of how well the compiler can vectorize the per-planet update and how large the resulting performance difference is, depending on the architectures under inspection. Additionally, the results should show whether a SoA layout is a precondition for effective SIMD vectorization as has been previously stated in Section 2.3. In order to achieve this, this scenario features the entire calculation of gravitational pull (see Section 3.1) and sets the payload size deliberately to `0` bytes. Otherwise, the memory overhead could distort the results and would not allow proper conclusions from the measurements.

Parametrization In this scenario, the number of planets is the only parameter being varied. It is swept from 1 planet up to 30 000, while the payload remains fixed at 0 bytes and the number of warmup and measurement iterations as well as the number of samples, i.e., repeated measurements, remain constant. Varying solely the planet count enables one to isolate the performance impact of scaling the arithmetic workload linearly, as the memory layout and footprint of a single planet remain identical throughout the sweep. Any difference between the architectures can therefore be attributed to how efficiently each of them executes the same arithmetic. Lastly, the sweep is performed in fine-grained steps rather than in orders of magnitude. This is deliberate, as this approach causes threshold effects to remain visible instead of being averaged away between two distant measurements.

Update Logic Equation 3.1 is translated directly into code. For every planet, the direction vector towards the center is obtained by negating the planet's position, from which the squared distance follows. The acceleration is then applied along the normalized direction. Lastly, velocity and position are updated over the fixed time step `dt`. Listing 3.1 shows the hot loop body being measured for the AoS and PC architectures. The sole difference between the two is the usage of `Box<T>` in the PC architecture, which enables the isolation of the effect of the usage of pointers versus direct memory access.

```
for planet in planets.iter_mut() {
    // calculates acceleration
    let direction = -planet.position;
    let r_squared = direction.length_squared();
    let a_scalar = G / r_squared;
    let a = a_scalar * direction.normalize();

    // updates velocity and position
    planet.velocity += a * dt;
    planet.position += planet.velocity * dt;
}
```

Listing 3.1: Update loop of the AoS and PC architectures

While there is a data dependency between the velocity and position of a planet, i.e. the velocity must be written before position can read it, it is not in the way of vectorization as it is contained within a planet. Whether SIMD vectorization occurs therefore depends on the respective memory layout.

```
for i in 0..planets.positions_x.len() {
    let px = planets.positions_x[i];
    let py = planets.positions_y[i];
    let pz = planets.positions_z[i];
    // calculates acceleration
    let r_sq = px * px + py * py + pz * pz;
    let r_recip = 1.0 / r_sq.sqrt();
    let a_scalar = G / r_sq;
    let nx = -px * r_recip;
    let ny = -py * r_recip;
    let nz = -pz * r_recip;
    // updates velocity and position
    planets.velocities_x[i] += nx * a_scalar * dt;
    planets.velocities_y[i] += ny * a_scalar * dt;
    planets.velocities_z[i] += nz * a_scalar * dt;
    planets.positions_x[i] += planets.velocities_x[i] * dt;
    planets.positions_y[i] += planets.velocities_y[i] * dt;
    planets.positions_z[i] += planets.velocities_z[i] * dt;
}
```

Listing 3.2: Update loop of the SoA architecture

The SoA implementation performs the same computation, but on six of the seven vectors from Section 3.2, as the payload remains untouched. The difference to AoS and PC is therefore not what is computed, but which values lie adjacent in memory. There, the three coordinates of a single vector are adjacent and consecutive planets are separated by the stride of the entire struct. Here, the x, y, and z values of consecutive planets are adjacent and thus form homogeneous data streams that can be vectorized, at the cost of markedly higher code complexity. To keep the arithmetic identical across architectures, `normalize()` is mirrored through `r_recip`, as it is internally implemented as a multiplication by the reciprocal of the length rather than as three separate divisions.

```
for (mut pos_slice, mut vel_slice) in query.contiguous_iter_mut(&mut world).unwrap() {
    for (pos, vel) in pos_slice.iter_mut().zip(vel_slice.iter_mut()) {
        // calculates acceleration
        let direction = -pos.0;   // unwraps the Position component to its inner Vec3A
        let r_squared = direction.length_squared();
        let a_scalar = G / r_squared;
        let a = a_scalar * direction.normalize();
        // updates velocity and position
        vel.0 += a * dt;
        pos.0 += vel.0 * dt;
    }
}
```

Listing 3.3: Update loop of the Bevy architecture with contiguous component slices

Finally, the Bevy architecture performs the same arithmetic and differs only in how the components are reached. As described in Section 3.2, `contiguous_iter_mut` yields slices rather than individual entities, over which the inner loop then iterates directly. Here, the trade-off again is between performance and code-complexity. Unlike the SoA architecture however, Bevy may occupy what could best be called a middle ground.

Verification While substantially better performance of the DOP-based architectures would suggest that vectorization has taken place, it would not by itself prove it. The generated x86-64 assembly is therefore inspected to determine whether the compiler emitted scalar instructions (`vaddss`, `vmulss`) or their packed SIMD counterparts (`vaddps`, `vmulps`), and whether 128-bit `xmm` or 256-bit `ymm` registers are used, with examples being present within the analysis in Section 4.2. The inspection uses `cargo-show-asm` [16], which interleaves assembly with the corresponding Rust source lines, and IDA Free [11], whose control flow representation makes the hot loop easier to identify. Vectorization can only occur if the compiler is permitted to emit the required instruction set extensions, enabled here by compiling with `-C target-cpu=native` (further build details in Section 4.1).

3.4 Scenario 2: Memory-Bound

As has been established in Section 2.1, cache pollution is the phenomenon when an object's data fills the CPU cache with data that is not relevant to the operation. In this scenario, this additional yet irrelevant data is simulated by the payload field of the planet structs, which is never read and serves solely to increase the size of a planet. The goal is to measure how strongly this unused data affects execution time, and from which object size onwards it begins to do so. As the intent is to have this scenario memory-bound, the calculation of gravitational pull is reduced to the update of a planet's position based on its current velocity, so as not to distort the results of benchmark runs with little payload. Conceptually, the planets do not rotate around a large mass anymore but wander through space in straight lines. Scenario 1 and Scenario 2 are thus complementary: the former minimizes the payload to measure the arithmetic, while the latter minimizes the arithmetic to measure the memory footprint's impact.

Parametrization In contrast to Scenario 1, the planet count remains fixed at 10 000, while the payload is the only parameter being varied, sweeping from 0 bytes up to 20 000 bytes. The number of warmup and measurement iterations as well as the samples per invocation again remain constant. The payload is deliberately declared as an array of fixed length rather than as a heap-allocated vector, so that it is stored within the struct itself and thus actually increases the size of a planet. In contrast, a vector would merely add a small, fixed-size handle to the structs size and thus leave the struct small. As the array's length is part of its type and thus must be known at compile-time, it requires a re-compilation for every payload size (see Section 3.6). The sweep is again performed in fine-grained steps, as the effects of cache conflict misses (see Section 2.2) are expected to appear at specific object sizes rather than gradually, and would be averaged away between two distant measurements.

Update Logic The core difference between architectures is now not directly visible in code anymore, as the differences are located in the underlying memory layout. For the PC and AoS architectures, the payload is interleaved with the position and velocity within each planet struct. Since the cache operates on fixed-size cache lines, every line fetched for the position and velocity also pulls in adjacent payload bytes, wasting precious cache bandwidth on irrelevant data. When performing an operation solely on the position and velocity vectors, only these relevant vectors may be loaded with the payload remaining untouched. Thus, the cache is filled only with relevant data, leading to potentially substantial performance improvements.

```
// PC and AoS
for planet in planets.iter_mut() {
    planet.position += planet.velocity * dt;
}
// SoA
for i in 0..planets.positions_x.len() {
    planets.positions_x[i] += planets.velocities_x[i] * dt;
    planets.positions_y[i] += planets.velocities_y[i] * dt;
    planets.positions_z[i] += planets.velocities_z[i] * dt;
}
// Bevy
for (mut pos_arr, mut vel_arr) in query.contiguous_iter_mut(&mut world).unwrap() {
    for (pos, vel) in pos_arr.iter_mut().zip(vel_arr.iter_mut()) {
        pos.0 += vel.0 * dt;
    }
}
```

Listing 3.4: Update loop of all four architectures in Scenario 2

Listing 3.4 shows the reduced complexity of the update loop of all four architectures. The SoA architecture uses a `PlanetsSoA` struct which contains seven vectors, six of which represent the x, y and z values of position and velocity of all planets, with the seventh containing the payload of all planets. As the payload is not required for the update on position, the array is simply not accessed and the cache is thus kept full with the relevant coordinate data. The Bevy architecture follows the same principle, querying only for the position and velocity components and never accessing the payload component. Bevy's idiomatic iterator (`query.iter_mut`) closely resembles the OOP update loop and would offer comparable syntactic simplicity. Here, however, the `contiguous_iter_mut` variant is used instead: while both iterators traverse the same contiguous `Table` storage, only `contiguous_iter_mut` exposes it as `&mut [T]` slices. This slice-based access allows the compiler to vectorize the loop and thus increases comparability between the Bevy and SoA architectures, at the cost of a more complex and verbose nested loop.

Verification While again the assembly code is inspected to verify the existence of vectorization, the main focus now lies on the cache miss rate, which is gathered with AMD μProf [1]. The cache miss rate would clearly indicate the cache being filled with irrelevant data, as a large payload size in relation to the relevant data leads to substantially more cache misses. As the effect of cache inefficiencies and cache conflict misses is highly dependent on the exact payload size used and different payload sizes necessitate separate binaries, it is not feasible to inspect every single binary created. Instead, the binaries at and around strong variations in performance are evaluated for SIMD instructions and cache miss rate to confirm the variations occur due to the aforementioned reasons.

3.5 Scenario 3: Allocation and Deallocation

With Scenario 1 measuring the cost of arithmetic and Scenario 2 the cost of a growing memory footprint, Scenario 3 measures the cost of repeatedly allocating and deallocating that data. This scenario simulates the repeated creation and destruction of planets. Practical implementations of this principle can be found with particle systems and projectiles in video games or simply short living entities in simulations. The core question is how expensive this repeated creation and destruction is, and how it scales with payload size and chunk size. Here, the chunk size declares how many planets are created and destroyed within a single loop. As the focus

is on memory management, the arithmetic in the update loop is left out completely. Additionally, two subvariants of this scenario are inspected, namely by how much performance differs depending on whether the data of destroyed planets is reused after destruction or whether it is discarded.

Parametrization This scenario, contrary to Scenario 1 and 2, varies two parameters: payload size and chunk size. This results in a 2D parameter sweep which acts as the basis for a heatmap. The payload is swept from 0 bytes to 25 000 bytes in less fine-grained 1000 byte steps, as the exact behaviour of cache conflict effects is already covered in Scenario 2 and the scaling behaviour is the main property under inspection. Chunk Size is swept from 25 to 1000 in steps of 25. As with Scenario 2, every change in payload requires re-compilation.

Update Logic As was previously stated, a distinction is made between discarding the popped objects and reusing their data. The discarding of data best represents conventional particle systems where particles are created, worked with and then destroyed. The reuse of data better represents entities in more complex simulations, where the data may want to be read after destruction. Listing 3.5 shows this difference in code, where in the reuse case the `planets.pop()` function is surrounded by a `black_box`, which in this case simulates the usage of the planet's data. On the contrary, `let _ = planets.pop();` discards the data immediately, simulating a particle system. This distinction and the usage of `black_box` occurs analogously in the SoA and Bevy architectures. For the creation of the planets, the coordinates are filled with 0.

```
if reuse {
    for _ in 0..chunk_size {
        black_box(planets.pop());
    }
} else {
    for _ in 0..chunk_size {
        let _ = planets.pop();
    }
}
```

Listing 3.5: Deallocation of planets in PC and AoS architectures

In the reuse case, the PC and AoS architectures differ notably. Popping a planet from the AoS vector moves the whole struct out of storage, i.e., an implicit `memcpy` of the 24 bytes of coordinates plus the full payload, so its cost grows with the payload size. In the PC architecture, the same pop

moves only the 8 byte `Box` pointer while the data stays on the heap, making the cost payload-independent. This only shows in the reuse case, as `black_box` forces the popped value to materialize; in the discard case the compiler is free to optimize the move away. This advantage is offset by allocation cost. In the PC architecture, every `push()` triggers a `malloc` and every `pop()` a `free`, for each element in every cycle. The AoS vector instead keeps its capacity across the cycles and thus barely allocates once it has grown. Therefore, PC trades a cheap pop against constant `malloc` and `free` overhead, and which effect dominates is expected to depend on payload and chunk size.

The difference between the AoS and SoA architectures lies in the AoS architecture requiring only a single `push()` and `pop()` per planet, whereas the SoA architecture requires seven such calls due to the split management of values across seven vectors. Bevy provides its own `spawn()` and `despawn()` functions; in the reuse case, however, explicit calls to `get::<Position>()`, `get::<Velocity>()` and `get::<Payload>()` are necessary, as `despawn()` does not return the component data of the removed entity.

Verification To verify the results, the assembly code is inspected to confirm that allocations and deallocations have been carried out as intended and have not been optimized away by the compiler. For this, `cargo show-asm` [16] is used. Additionally, the usage of the `memcpy` instruction is inspected in the AoS architecture, to confirm the hypothesis of it requiring expensive memory movements instead of simple pointer popping. A heatmap is used to visualize the runtimes and more easily discern if any threshold effects or performance anomalies occur.

3.6 Execution and Configuration

All architectures and benchmarks are compiled into a single executable located at `../Benchmarks/target/`. An individual benchmark run is configured almost entirely through Command Line Interface (CLI) arguments, with the parsing of the commands being handled by the `clap` crate [15]. Table 3.1 lists the available arguments.

The program itself does not contain the functionality of parameter sweeping, i.e., varying parameter values within defined ranges. Instead, PowerShell and bash scripts are provided which repeatedly execute the benchmark with varying values. This keeps the measured code free from

Short	Long	Default	Description
`-b`	`--benchmark`	required	Scenario: `s1`, `s2`, `s3` or `reset`
`-p`	`--planets`	10 000	Number of planets created during setup
`-w`	`--warmups`	2500	Warmup iterations, results discarded
`-i`	`--iterations`	5000	Measured iterations per sample
`-s`	`--samples`	5	Number of samples per invocation
`-c`	`--chunksize`	100	Elements added and removed per iteration (S3)
`-r`	`--reuse`	`false`	Enables the reuse variant (S3)
`-d`	`--debug`	`false`	Prints a runtime summary to the console

Table 3.1: CLI arguments for configuring the benchmark executable

any sweeping logic and makes every measurement reproducible from its arguments alone. In addition, the scripts recompile the benchmarks with varying payload sizes for Scenarios 2 and 3, as the payload size is a compile-time constant and thus must be defined before the execution of the benchmarks.

A single invocation executes all four architectures in sequence. For every architecture and for every sample, the corresponding benchmark function is called twice: once with the number of warmup iterations and once with the number of benchmark iterations. Calling the same function for both warmup and benchmark iterations ensures that the cache is in a comparable state when the measurements begin. A typical invocation is shown in Listing 3.6.

```
cargo run --profile o3 -- -b s3 -w 1000 -i 5000 -s 5 -c 500 -r -d
```

Listing 3.6: A benchmark invocation for Scenario 3 via the command line

Measurement results are appended to a Comma-Separated Values (CSV) file per architecture and scenario. For example, the above invocation would append the measurements to the files `bevy.csv`, `soa.csv`, `etc.` located at `../Benchmarks/diagrams/s3/csv/`. Each line contains an entry for the respective architecture, planet count, number of iterations, chunk size, reuse of deallocated data in Scenario 3, payload size in bytes as well as the elapsed time in nanoseconds. In order to delete the generated CSV files, the `reset` value for the `benchmark` argument may be used. This acts as a way to create a clean slate and prevent appending measurements to those of previous benchmarks, should the CSV files not have been moved yet.

Chapter 4

Evaluation

Chapter 3 defined the benchmark design and introduced the four architectures under comparison, as well as the three scenarios. This chapter presents and interprets the measurements obtained from the benchmark runs. First, the setup and environment are documented. The subsequent sections then present the measurements of the three scenarios. A final discussion draws together the trends observed across all scenarios, followed by a discussion of the limitations of this evaluation.

4.1 Methodology

Hardware All benchmark runs were performed on a desktop machine equipped with an AMD Ryzen 7 5800X processor and 32 GB of DDR4-3200 memory. The processor provides eight physical cores, each exposing two hardware threads through Simultaneous Multithreading (SMT). Its L1 data cache is 32 KiB in size, 8-way set associative, and organized in 64 byte cache lines. The processor further supports the AVX2 instruction-set extensions, which the vectorized code paths rely on.

Environment The benchmarks were run on Ubuntu 26.04 LTS (kernel 7.0.0-28-generic). Two settings were configured in the BIOS: Precision Boost Overdrive (PBO) and C-states were disabled — the former to prevent frequency boosting, the latter to keep the benchmark core from entering low-power idle states. The remaining modifications were made at runtime by a setup script. Core Performance Boost (CPB), AMD's counterpart to Intel's Turbo Boost, was disabled, and the frequency of all cores was locked to 3.6 GHz, so that the clock stays fixed. The SMT sibling sharing the physical core with logical CPU 2 (logical CPU 10) was taken offline, so that no tasks running on logical CPU 10 would consume resources of core 2, to which the benchmark was pinned via `taskset`.

Build/Compiler The benchmarks were compiled with the rustc 1.95.0 compiler. Rust supports compilation profiles, which let the programmer declaratively define a set of compilation options. For the benchmark program, the profile `run` was defined, which inherits from the standard `release` profile, explicitly disables debug information, and sets the optimization level to 3. Two further compiler flags were set in the execution scripts: `-C target-cpu=native` and `-C link-arg=-fuse-ld=mold`. The former targets the CPU and enables the AVX2 supported by the Zen 3 architecture, the latter selects the mold linker, which shortens linking time. Because a linker does not alter the generated machine code, this choice has no effect on the run time of the architectures.

Measurement Protocol To further diminish the impact of OS jitter, a setup with iterations and samples is used. In a given scenario, the four architectures perform a number of warmup runs, whose results are discarded, followed by a number of measured iterations. The warmup lets the caches reach a steady state before timing begins. The architectures are measured in the order of PC $\rightarrow$ AoS $\rightarrow$ SoA $\rightarrow$ Bevy. This whole procedure is executed `samples` times.

Scenario	Warmups	Iterations	Samples	Measured Iterations
S1	100	50	10	500
S2	200	200	5	1000
S3	100	50	5	250

Table 4.1: Per-scenario measurement parameters.

As an example, consider the measurement parameters for Scenario 2 in Table 4.1. Instead of each architecture being measured in one block of 1000 iterations, the architectures are measured in interleaved, smaller blocks, namely 5 samples of 200 iterations in the case of Scenario 2. Thus, a larger system interrupt would be more likely to affect multiple or all architectures instead of only one. Each sample stores the run time over its `iterations` update steps as a single `time_ns` value in the CSV. The division by the iteration count, resulting in the time per iteration, as well as the averaging over the samples are performed when the plots are generated, which display the mean per-iteration time across all samples.

4.2 Scenario 1: Compute-Bound

Scenario 1 measures the performance when the computational workload becomes the bottleneck, i.e., this scenario represents the case where performance is compute-bound. First, every architecture calculates the acceleration of each planet towards the origin, then the velocity and position of each planet are updated. The only parameter being swept is the planet count. All architectures calculate the same physical update. The difference lies in how the data is laid out in memory, and as a consequence, how that update is expressed in code.

Scaling Behavior As is clearly visible in Figure 4.1, all four architectures scale linearly. However, there are clear differences in the performance between architectures. The most performant one in calculating the positional update of 30 000 planets is, with a big margin, the SoA architecture, only taking 22.62 µs per iteration. In second place is the other data-oriented approach, the Bevy architecture, requiring 106.34 µs per iteration. Then follows the AoS architecture with 138.32 µs and concluding with the PC architecture with 213.91 µs.

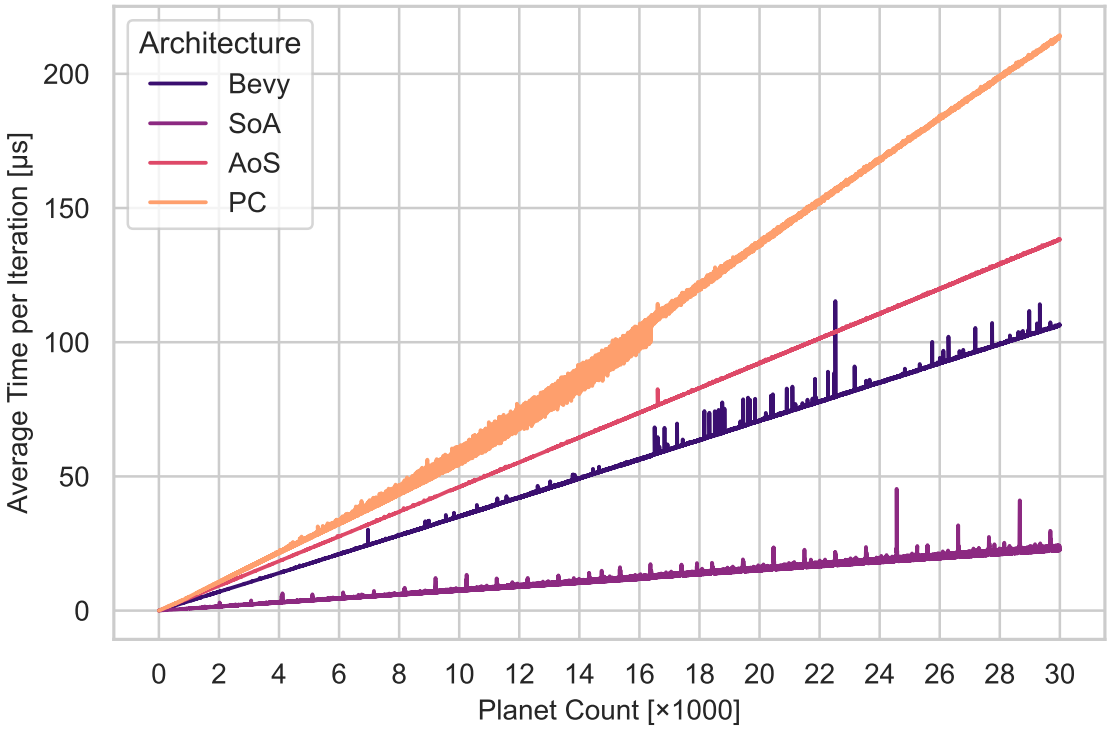


Figure 4.1: Scenario 1 (Compute-Bound): Average time per iteration versus planet count

To be noted is the wide yet relatively constant variation in measurements regarding the PC architecture, which lessens after 16 384 planets. Additionally, a number of spikes in execution time are visible in the measurements of the Bevy and SoA architectures. However, these spikes do not affect the overall trend.

Assembly Analysis To verify the results visible in Figure 4.1, the assembly code of the `bench_s1_pc`, `bench_s1_aos`, `bench_s1_soa`, and `bench_s1_bevy` functions is inspected for the usage of wider `ymm` registers and packed instructions. Note that these files can reach several hundred lines of code. Therefore, Figure 4.2 mainly focuses on arithmetic of the hot loop, which is the calculation of acceleration and its application to velocity and position. The machine code instructions are reordered to match the source order. Regarding the SoA architecture, for the per-component velocity and position update, only the x-component is shown.

SoA

```
; let px = planets.pos_x[i];
vmovups ymm5, ymmword ptr [r10 + 4*rax]

; let r_sq = px * px + py * py + pz * pz;
vmulps  ymm8, ymm5, ymm5
vmulps  ymm9, ymm6, ymm6
vaddps  ymm8, ymm8, ymm9
vmulps  ymm9, ymm7, ymm7
vaddps  ymm8, ymm8, ymm9
; let r_recip = 1.0 / r_sq.sqrt();
vsqrtps ymm9, ymm8
vdivps  ymm9, ymm3, ymm9

; let a_scalar = G / r_sq;
vdivps  ymm8, ymm4, ymm8
; let nx = -px * r_recip;
vmulps  ymm5, ymm9, ymm5

; planets.vel_x[i] += nx * a_scalar * dt;
vmulps  ymm5, ymm8, ymm5
vmulps  ymm5, ymm0, ymm5
vsubps  ymm5, ymm10, ymm5
; planets.pos_x[i] += planets.vel_x[i] * dt;
vmulps  ymm5, ymm0, ymmword ptr [r11 + 4*rax]
vaddps  ymm5, ymm5, ymmword ptr [r10 + 4*rax]
vmovups ymmword ptr [r10 + 4*rax], ymm5
add     rax, 8
```

AoS

```
; let dir = -planet.pos;
vmovsd   xmm5, qword ptr [rax + rsi + 12]
vmovss   xmm3, dword ptr [rax + rsi + 20]

; let r_squared = dir.length_squared();
vmulps    xmm6, xmm5, xmm5
vmulss    xmm4, xmm3, xmm3
vmovshdup xmm7, xmm6
vaddss    xmm6, xmm6, xmm7
vaddss    xmm4, xmm6, xmm4

; let a_scalar = G / r_squared;
vdivss    xmm6, xmm1, xmm4
; let a = a_scalar * direction.normalize();
sqrtss    xmm4, xmm4
vdivss    xmm4, xmm2, xmm4
vmulps    xmm7, xmm7, xmm5
vmulps    xmm6, xmm0, xmm7

; planet.velocity += a * dt;
vmovsd    xmm7, qword ptr [rax + rsi]
vsubps    xmm6, xmm7, xmm6
vmovlps   qword ptr [rax + rsi], xmm6
; planet.pos += planet.vel * dt;
vmulps  xmm6, xmm0, xmm6
vaddps  xmm5, xmm5, xmm6
vmulss  xmm4, xmm9, xmm4
vmovlps qword ptr [rax + rsi + 12], xmm5
vaddss  xmm3, xmm3, xmm4
vmovss  dword ptr [rax + rsi + 20], xmm3
add     rsi, 24
```

Figure 4.2: ASM arithmetic: SoA versus AoS

Figure 4.2 shows that the SoA architecture is able to work with the wider 256 bit `ymm` registers, while the AoS architecture is forced to operate on the narrower 128 bit `xmm` registers. Additionally, the SoA architecture is able to vectorize, relying heavily on packed instructions such as `vmulps` and `vsqrtps`. Finally, the `add rax, 8` advances the index by eight elements, so the next eight planets are processed.

In contrast, the AoS architecture cannot vectorize fully, falling back to scalar instructions such as `vdivss` and `sqrtss`. Additionally, the stride `add rsi, 24` equals the size of the `PlanetAoS` struct in bytes, confirming that only one planet is processed per iteration rather than eight.

AoS

```
1  ; planet.position (direct load)
2  vmovsd xmm5, qword ptr [rax + rsi + 12]
3  ; ... identical arithmetic ...
4  add    rsi, 24
```

PC

```
1  ; planet.position (pointer load + deref)
2  mov    rdi, qword ptr [rcx + rsi]
3  vmovsd xmm5, qword ptr [rdi + 12]
4  ; ... identical arithmetic ...
5  add    rsi, 8
```

Figure 4.3: ASM Load sequence: AoS versus PC

Regarding the low performance of the PC architecture, the machine code shows that the arithmetic between the AoS and PC architectures is identical. As is visible in Figure 4.3, the main difference lies in how the planet's data is accessed, with AoS able to load data directly and PC having to fetch the data through the pointer address. This is further confirmed by the stride length of 24 for AoS, which is the `PlanetAoS` struct's size in bytes, whereas the stride length for PC is 8, a pointer's size in bytes.

The Bevy architecture does not compile to a single, isolated hot loop, as the chunked entity iteration produces several separate loops, which is why no representative excerpt is shown. Its arithmetic is partially vectorized, as some `ymm` registers are present in comparison to the object-oriented architectures, which use only `xmm` registers. However, the usage of these `ymm` registers is interwoven with shuffle instructions, e.g., `vshufpd`, that assemble the lanes, reflecting only partial vectorization compared to SoA, which in contrast can cleanly operate on eight planets in at once. This is likely a consequence of the `Vec3A` layout, which stores each entity's x, y, and z coordinates together unlike the SoA architecture. However, the usage of the ECS still enables a considerable performance improvement compared to the object-oriented architectures, as it enables at least partial vectorization and usage of wider registers.

Interpretation To summarize, memory layout dictates whether vectorization can occur or not. As was seen, the SoA architecture was able to fully exploit the usage of wide registers and packed instructions due to the component arrays, whereas the object-oriented approaches employing interleaved memory layouts were not able to vectorize. Bevy lies in the middle, achieving partial vectorization. Additionally, the effect of memory locality was isolated between the AoS and PC architectures, demonstrating that vectorization is not the only aspect that can improve performance. To conclude, in a compute-bound scenario, the benefit of a data-oriented approach lies not in performing each calculation faster (the arithmetic is identical) but in how many planets are processed per instruction. The performance of Bevy shows that a practical, ECS-based approach can outperform the object-oriented layouts in this compute-bound case, though the necessity to use structs such as `Vec3A` impedes the full speed-up that a pure SoA architecture would achieve. Thus, programs do not adhere exactly to either DOP or OOP, but lie on a spectrum spanned between the two.

4.3 Scenario 2: Memory-Bound

In contrast to Scenario 1 which measured the performance difference between architectures when compute-bound, Scenario 2 measures the performance impact when computational workload is negligible, but large amounts of data are accessed. The hot loop now solely consists of a positional update based on existing velocity, however, with varying payload size. The object-oriented architectures carry this payload within the `PlanetAoS` struct, while the SoA architecture holds the payload in a separate vector and the Bevy architecture holds it in a separate `Payload` component. Thus, the SoA architecture is able to only iterate over the relevant vectors, whereas in the Bevy architecture the `Query` of the `Position` and `Velocity` components explicitly leaves out the `Payload` component.

Scaling Behavior As was stated in Section 2.4, an ECS benefits greatly from the separation of relevant and irrelevant data, which is clearly visible in the results in Figure 4.4. While the execution time of the PC and AoS architectures scales directly with payload size, the SoA and Bevy architectures remain constant. The order of performance follows suit with Scenario 1, with SoA being in first place with an average time per iteration of 2.32 µs. In second place is the Bevy architecture with 5.03 µs, followed by the AoS architecture with 26.69 µs and the PC architecture with 37.94 µs.

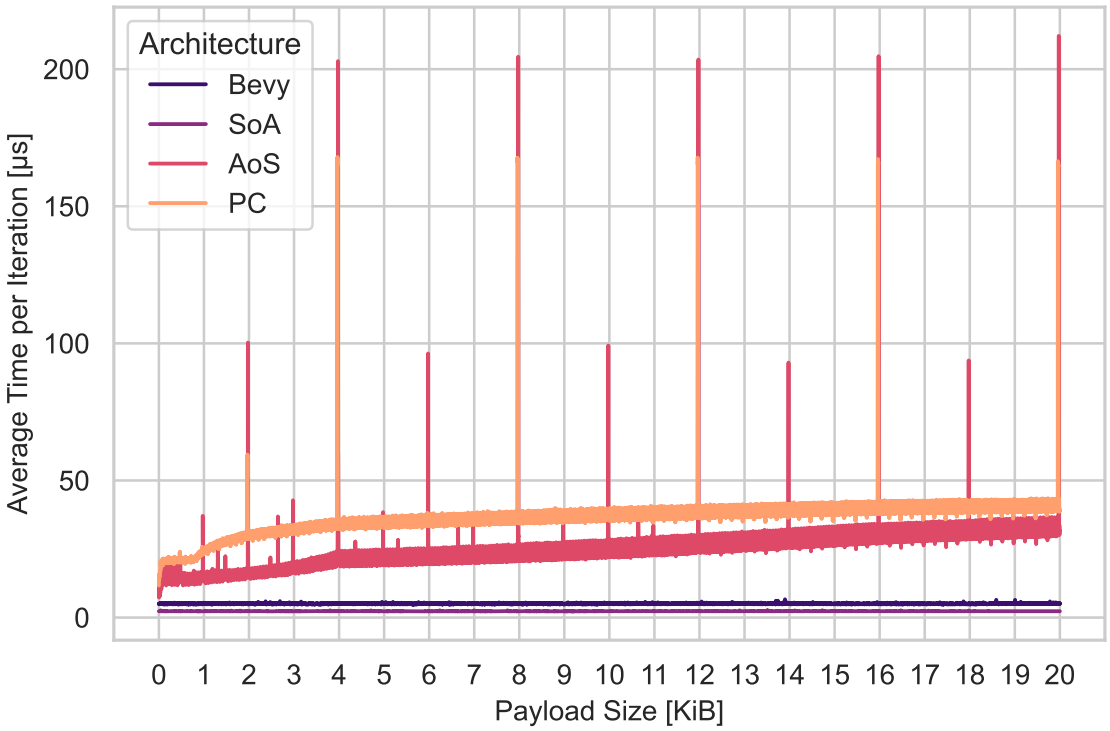


Figure 4.4: Scenario 2 (Memory-Bound): Average time per iteration versus payload size

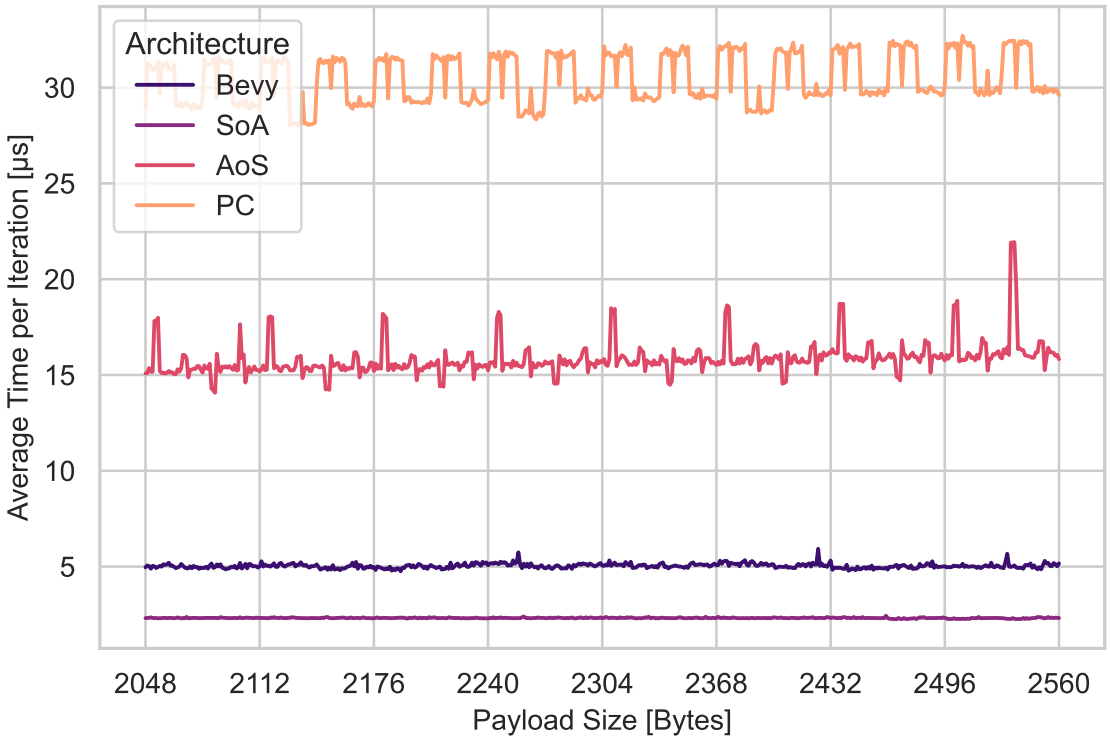


Figure 4.5: Scenario 2 (Memory-Bound): Pattern in execution time of PC and AoS architectures

While the data-oriented architectures feature smooth curves in Figure 4.4, the object-oriented architectures feature a distinct pattern of spikes. Both architectures possess large spikes in execution time at multiples of 4 KiB, whereas the AoS architecture, in addition, features medium spikes at 2 KiB, 6 KiB and 10 KiB and so on, as well as small spikes when the payload size is an odd multiple of 1 KiB.

Additionally, what originally appeared as jitter in Figure 4.4 is another pattern in execution times, as Figure 4.5 clearly shows. Especially for the AoS architecture, a clear period of 64 bytes can be noted when looking at the taller spikes of the micro-pattern.

	PC		AoS		SoA		Bevy	
Payload	CPI	L1 [%]	CPI	L1 [%]	CPI	L1 [%]	CPI	L1 [%]
998	1.079	21.19	1.474	26.67	0.786	32.25	0.644	18.86
1010	1.049	21.23	0.709	20.88	0.767	31.80	0.659	18.93
2022	1.267	25.52	6.426	77.11	0.679	31.66	0.579	18.86
2034	1.284	26.04	0.721	21.10	0.976	37.18	0.564	18.77
4070	1.552	29.97	12.680	96.79	0.771	31.66	0.726	18.94
4082	1.754	33.10	0.812	23.83	0.750	31.57	0.553	19.10

Table 4.2: CPI and L1 data cache miss ratio per architecture. Payload pairs represent spikes at 1024 B, 2048 B and 4096 B

Interpretation

4.4 Scenario 3: Allocation and Deallocation

- Results and Analysis
- drop vs reuse

4.5 Discussion

- High Level Overview and Comparison of findings

4.6 Limitations

Chapter 5

Related Work

- Winqvist
- Nyberg
- Related Work aus Nyberg (1-2)

Chapter 6

Future Work

- Programming/Code complexity
- Full scale programs for inspection

Chapter 7

Conclusion

Bibliography

[1] Advanced Micro Devices. 2026. *Amd µProf*. Version 5.3. Advanced Micro Devices Inc. Retrieved 07/30/2026 from https://www.amd.com/de/developer/uprof.html.

[2] Tony Albrecht. 2009. Pitfalls of Object Oriented Programming. Presented at Game Connect: Asia Pacific (GCAP) 2009. Sony Computer Entertainment Europe, Research & Development Division, (2009). Retrieved 07/27/2026 from https://www.gamedevs.org/uploads/pitfalls-of-object-oriented-programming.pdf.

[3] Bevy Project Contributors. 2026. *Bevy Quick Start Guide: ECS*. Bevy Engine. Retrieved 07/27/2026 from https://bevy.org/learn/quick-start/getting-started/ecs/.

[4] Bevy Project Contributors. 2026. *`bevy::ecs::storage::Table` — Bevy API Documentation*. Bevy Engine. Retrieved 07/27/2026 from https://docs.rs/bevy/0.19.0/bevy/ecs/storage/struct.Table.html.

[5] Scott Bilas. 2002. A Data-Driven Game Object System. Presented at the Game Developers Conference (GDC) 2002. Gas Powered Games, (2002). Retrieved 03/11/2026 from https://www.gamedevs.org/uploads/data-driven-game-object-system.pdf.

[6] Ulrich Drepper. 2007. What Every Programmer Should Know About Memory. Technical report. Red Hat, Inc., (November 2007). Retrieved 03/06/2026 from https://people.freebsd.org/~lstewart/articles/cpumemory.pdf.

[7] Richard Fabian. 2018. *Data-oriented Design: Software Engineering for Limited Resources and Short Schedules*. Cited from the free online edition. Richard Fabian. ISBN: 978-1-916478-70-1. Retrieved 07/27/2026 from https://dataorienteddesign.com/dodbook.pdf.

[8] Toni Härkönen. 2019. *Advantages and Implementation of Entity-Component-Systems*. Bachelor's thesis. Tampere University, (April 2019). Retrieved 07/27/2026 from https://urn.fi/URN:NBN:fi:tty-201905231735.

[9] Cameron Hart. 2026. *`glam::Vec3A` — API Documentation*. Retrieved 07/27/2026 from https://docs.rs/glam/0.32.1/glam/f32/struct.Vec3A.html.

[10] John L. Hennessy and David A. Patterson. 2011. *Computer Architecture: A Quantitative Approach*. (5th ed.). Morgan Kaufmann, Waltham, MA. ISBN: 978-0-12-383872-8.

[11] Hex-Rays SA. 2026. *IDA Free*. Version 9.4. Hex-Rays. Retrieved 07/29/2026 from https://hex-rays.com/ida-free.

[12] Jenya705. 2026. Contiguous access (`Query::contiguous_iter_mut`). Bevy Engine. Retrieved 08/07/2026 from https://github.com/bevyengine/bevy/pull/21984.

[13] Steve Klabnik and Carol Nichols. 2023. *The Rust Programming Language*. (2nd ed.). Cited from the free online edition. No Starch Press, San Francisco, CA. ISBN: 978-1-7185-0310-6. Retrieved 03/07/2026 from https://doc.rust-lang.org/book/.

[14] Daniel Kleppner and Robert J. Kolenkow. 2014. *An Introduction to Mechanics*. (2nd ed.). Cambridge University Press, Cambridge. ISBN: 978-0-521-19811-0. Retrieved 08/06/2026 from https://archive.org/details/KleppnerD.KolenkowR.J.IntroductionToMechanics2014.

[15] Kevin Knapp and clap Contributors. 2026. *`clap` — Command Line Argument Parser for Rust*. Retrieved 07/22/2026 from https://docs.rs/clap/4.6.0/clap/.

[16] cargo-show-asm Contributors pacac. 2026. *`cargo-show-asm` — Display Assembly Generated for Rust Source Code*. Version 0.2.57. Retrieved 07/29/2026 from https://docs.rs/cargo-show-asm/0.2.57.

[17] Simon J. Pennycook, Simon D. Hammond, Steven A. Wright, J. A. Herdman, I. Miller, and Stephen A. Jarvis. 2013. An investigation of the performance portability of OpenCL. *Journal of Parallel and Distributed Computing*, 73, 11, 1439–1450. DOI: 10.1016/j.jpdc.2012.07.005.

[18] Patrick Redmond, Jonathan Castello, José Manuel Calderón Trilla, and Lindsey Kuper. 2025. Exploring the Theory and Practice of Concurrency in the Entity-Component-System Pattern. *Proceedings of the ACM on Programming Languages*, 9, OOPSLA2, 1–28. DOI: 10.1145/3763050.

[19] Xinmin Tian, Hideki Saito, Serguei Preis, Eric N. Garcia, Sergey Kozhukhov, Matt Masten, Aleksei G. Cherkasov, and Nikolay Panchenko. 2013. Practical SIMD Vectorization Techniques for Intel® Xeon Phi Coprocessors. In *2013 IEEE International Symposium on Parallel & Distributed Processing, Workshops and PhD Forum*. IEEE, Cambridge, MA, USA, pp. 1149–1158. DOI: 10.1109/IPDPSW.2013.245.

[20] Wm. A. Wulf and Sally A. McKee. 1995. Hitting the memory wall: implications of the obvious. *ACM SIGARCH Computer Architecture News*, 23, 1, 20–24. ISSN: 0163-5964. DOI: 10.1145/216585.216588.

List of Tables

3.1 CLI arguments for configuring the benchmark executable . . 20

4.1 Per-scenario measurement parameters. 22
4.2 CPI and L1 data cache miss ratio per architecture. Payload pairs represent spikes at 1024 B, 2048 B and 4096 B 28

List of Figures

2.1 Memory Layout Visualization: AoS vs. SoA (taken from [17]) 3
2.2 Visualization of the ECS pattern depicting (1) Entities, (2) Components and (3) Systems (taken from [18]) 6

4.1 Scenario 1 (Compute-Bound): Average time per iteration versus planet count 23
4.2 ASM arithmetic: SoA versus AoS 24
4.3 ASM Load sequence: AoS versus PC 25
4.4 Scenario 2 (Memory-Bound): Average time per iteration versus payload size 27
4.5 Scenario 2 (Memory-Bound): Pattern in execution time of PC and AoS architectures 27

List of Acronyms

AoS Array of Structures

CLI Command Line Interface

CPI Cycler per Instruction

CPB Core Performance Boost

CSV Comma-Separated Values

DOP Data-oriented Programming

ECS Entity Component System

GPR General Purpose Register

OOP Object-oriented Programming

PBO Precision Boost Overdrive

PC Pointer Chasing

SIMD Single Instruction, Multiple Data

SMT Simultaneous Multithreading

SoA Structure of Arrays